Web Scorer & Trust Analyzer

Overview

This project is a multi-source data scraping and trust scoring pipeline. It dynamically extracts metadata and content from Blogs, YouTube, and PubMed, calculates a weighted Trust Score, and generates abstractive machine-learning summaries.

1. Tools and Libraries Used

- Python: Core programming language.
- Flask: Framework used to serve the dynamic Glassmorphism web dashboard.
- BeautifulSoup4: HTML parsing and scraping for general blog articles.
- yt-dlp & youtube-transcript-api: Used to extract YouTube metadata and video transcripts.
- Biopython (Entrez): API toolkit used to search and fetch structured XML medical data from the NCBI PubMed database.
- HuggingFace Transformers (`sshleifer/distilbart-cnn-12-6`): Deep learning NLP model utilized for abstractive machine summarization.
- KeyBERT: NLP library utilized to automatically extract the most relevant topic keywords.
- langdetect: To detect the language of the scraped content (with custom fallbacks for Hinglish).

2. Scraping Approach

Our scraping architecture is highly modular, featuring dedicated scrapers for each source type:

- Blogs: Uses `requests` and `BeautifulSoup4` to parse standard HTML paragraph tags (`<p>`) and `meta` tags for author/date extraction.
- YouTube: Bypasses traditional scraping by leveraging `yt-dlp` for metadata (views, duration, upload date) and `youtube-transcript-api` to pull the actual spoken text transcript for content processing.
- PubMed: Interacts directly with the official Entrez NCBI API to cleanly extract authors, publication dates, abstracts, and citation counts without relying on brittle HTML parsing.

3. Trust Score Design

The Trust Score calculates the credibility of the source using a heuristic weighted function yielding a percentage (0-100%). The weights are dynamically assessed:

- Author Credibility (25%): Prioritizes recognized medical or academic institutions (e.g., clinics, universities).
- Citation Count (20%): Directly correlates with PubMed's citation indices (a highly cited paper achieves a maximum score).
- Domain Authority (20%): Heavily weights `.gov` and `.edu` domains (1.0), treats `youtube.com` as neutral (0.7), and penalizes standard `.com` blogs (0.5).
- Recency (20%): Content published within the last year scores highly, while content older than 5 years is penalized to ensure up-to-date information.
- Medical Disclaimer Presence (15%): Scans the text for legal disclaimers (e.g., "not a substitute for medical advice") which indicate a professional and cautious approach to health reporting.

4. Limitations

- YouTube Transcripts: The system relies on user-uploaded or auto-generated transcripts. If a video disables transcripts, the system must fall back to summarizing only the video description.
- JavaScript-Rendered Blogs: Standard `BeautifulSoup` cannot execute client-side JavaScript. Blogs that dynamically load their content via React/Angular without SSR might fail to scrape.
- Language Bias: The `distilbart` abstractive summarization model is trained primarily on English text. Non-English articles may yield lower-quality summaries or translation artifacts.

5. How to Run the Project

1. Install Dependencies: Ensure you have Python installed, then run:

```bash

pip install -r requirements.txt

```

2. Run Batch Pipeline: To scrape the hardcoded assignment links and generate the structured JSON file:

```bash

python main.py

```

(This will create the dataset in the `output/` directory).

3. Run Dynamic Web Dashboard: To start the local Flask server and visually analyze custom links:

```bash

python app.py

```

Navigate to `http://localhost:5000` in your browser.